Abstract

REPLACE THIS [This document provides a basic paper template and submission guidelines. Abstracts must be a single paragraph, ideally between 4–6 sentences long. Gross violations will trigger corrections at the camera-ready phase.]

1 Introduction

Finding an optimal path in a given situation can be modeled using tabular reinforcement learning (RL). In this regime of RL the state and actions spaces are discrete. Relatively low-dimensional "grid worlds" in this regime allow for both state and action spaces to be stored in their entirety. Here we will cover several methods for finding the optimal path through a certain environment.

In section 2 the theory relevant for the methods used is covered. This will be based on the work in [**?**]. In section 3 the different experiments applied to find the optimal path through this environment are covered and their results are presented. In section 4 these results are discussed. We end with a reflection in section 5

2 Theory

Let's cover some terms that will be used throughout this work. An agent will make a sequence of actions to traverse an environment and reach a goal. Such a problem is therefore also called a Sequential Decision Problem (SDP). Though in real life, humans often let their experience of previous decisions lead their following decisions this is not the case in Markov Decisions Processes (MDPs). In MDPs the next action taken is solely based on the information available at a given state. By tackling an SDP using an MDP allows us to straightforwardly model an agent's behaviour. Taking all past experiences into account, becomes complicated. To model an SDP as an MDP the following things are needed: the actions the agent can take (the action space A), the states the environment can be in (the state space S), the rewards associated with each state R_s, the probability to transition from one state (s) to another (s'), given an action (a) denoted by $T(s'|a, s)$ and a discount factor γ which decides how much future possible rewards are weighted into the decision making process. So how do we use this information to decide what are good actions to take? First we need to know what the return R means. It is given by the following equation:

$$R = \sum_{i=0}^{N} r_i \gamma^i \tag{1}$$

With N the number of steps taken by the agent and r_i the reward received at time step i. This leads us to the state-action value function $Q(s, a)$. This is the expected return R if a given policy $\pi(a|s)$ is used to decide what action to take after being in state s and taking action a. Or, in equation form

$$Q^{\pi}(s, a) = \mathbb{E}[R] \tag{2}$$

The goal is to find the optimal policy which maximises this expected return, or

$$\pi^{\star} = \arg\max_{a \in A} Q^{\star}(s, a) \tag{3}$$

grid worlds , Bellman?

3 Experiments

Let's first go over the environment that will be used throughout this report. The environment in which the agent is moving around is a 7x10 grid world. The agent starts at a starting position (0,3). A goal location is set at (7,3) and can be moved around. Reaching the goal state yields a reward of 100. Reaching any other state yields a reward of -1. In columns 3, 4, 5 and 8 there's a 90% change that the agent will be moved upwards one tile, due to a stochastically present wind. In columns 6 and 7 this results in an upwards movement of two tiles with similar probability.

3.1 Dynamic programming

A method of constructing the state-action value function $Q(s, a)$ is by using the Value Iteration algorithm (see Algorithm 1). This algorithm loops over both state and action spaces to iteratively improve the Q table until the state-actions values don't change much anymore (convergence is reached). The output of this algorithm is an optimal (within the constraints of the parameters used) Q table which can be used to traverse the environment. In a given state s, the next action a' is chosen based on the action that maximises the $Q(s, a')$ value. This is called a *greedy* policy. This algorithm requires the transition probability to be known from the environment and is therefore a model-based solution method for the MDP.

* Show the environment with the Q values at different stages in the VI alorithm. Describe what these plots mean

* How is V(s) calculated? Include the gamma term!!

* Describe the average reward per timestep

* How is the goal state handled?

* What happens when the goal location is changed?

`Figures/step1_VI_at_end.pdf`

Figure 1: After 16 iterations of value iteration, convergence is reached (when using $\gamma = 0.99$ and $\eta = 0.001$). For ealier stages of the algorithm see Figure 2 and Figure 3. The agent will move to the right, end up in the top-right corner due to the stochastic wind, then work its way to the bottom-right and then to the left and up towards the goal.

Algorithm 1 Tabular Q-value iteration

Input: n_{states}, n_{actions}, threshold $\eta \in \mathrm{R}_{>0}$
Initialise: A state-action value table Q with shape $(n_{\text{states}}, n_{\text{actions}})$ and $Q(s,a) = 0 \; \forall s \in S, a \in A$
Output: Optimal state-action value table $Q^\star$
repeat
  $\Delta \leftarrow 0$
  for each $s \in S$ **do**
    for each $a \in A$ **do**
      $x \leftarrow Q(s,a)$
      $\tilde{Q}(s,a) \leftarrow \sum_{s'}[p(s'|s,a) \cdot (r(s,a,s') + \gamma \cdot \max_{a'} Q(s',a'))]$
      $\Delta \leftarrow \max\Big(\Delta, |x - \tilde{Q}(s,a)|\Big)$
    end for
  end for
until $\Delta < \eta$

3.2 Exploration

Algorithm 2 Tabular Q-learning

3.3 Back-up: On-policy versus off-policy target

Algorithm 3 Tabular SARSA

Input: Number of timesteps, Exploration parameter ϵ, learning rate $\alpha \in (0,1]$, discount parameter $\gamma \in [0,1]$
Initialise: Q
Output: Optimal state-action value table $Q^\star$

3.4 Back-up: Depth of target

Algorithm 4 N-step Q-learning

Algorithm 5 Tabular Monte Carlo RL

4 Discussion

4.1 Dynamic programming

5 Conclusion

A Dynamic programming: supplementary graphs

Figures/step1_VI_at_begining.pdf

Figure 2: The environment as described in section 3 when initialising the Q table with zeros for all actions a and states s. The agent will start at the blue square with an S in the top-left. The goal is denoted by the green square with a +100 in the top. In this setting the agent will move around randomly until it's reached a maximum number of steps or the goal is reached

Figures/step1_VI_halfway.pdf

Figure 3: Similar to Figure 2, but now the value iteration algorithm (described in subsection 3.1 is halfway through its iterations (having performed 8 of the 16 iterations until convergence is reached). This is visible by the fact that not every state has an optimal action yet (the optimal action is denoted by an arrow in the direction of the optimal action). With this Q table the agent will move to the right, be moved upwards by the wind, then move down to the bottom-right and then go left and up towards the goal state.